Preliminary Report
Hybrid Cryptography Project

Brynja Eyfjörð Jónsdóttir (s221901)
Karítas Ósk Pálmadóttir (s175414)
Kristofer Birgir Hjörleifsson (s253156)

March 17, 2026

1 What We Have Done

Task 1 – ETSI Hybrid Constructions

We reviewed ETSI TR 103 966 and summarized the hybrid construction types it describes. For KEMs, the document presents three approaches with increasing security: concatenation only ($K = K_{\text{classical}} \parallel K_{\text{pq}}$), concatenation with simple KDF, and concatenation with full KDF (which additionally includes ciphertexts as KDF input to strengthen binding between components). For signatures, it presents concatenation only (separable), concatenation with identifiers (weak non-separability), and strong non-separability. We include compact formulas for each construction and a comparison table of advantages and disadvantages.

We connect this to our implementation by noting that the TLS 1.3 hybrid key exchange (X25519MLKEM768) follows the concatenation with full KDF pattern, since the TLS key schedule (HKDF) acts as the key combiner.

Task 2 – ECCG/ENISA Recommendations

We reviewed the ECCG *Agreed Cryptographic Mechanisms* document and identified which post-quantum algorithms are agreed for use in the EU: ML-KEM and FrodoKEM for key encapsulation, and ML-DSA, SLH-DSA, XMSS, and LMS for digital signatures. We list the recommended parameter levels (e.g., ML-KEM-768/1024, FrodoKEM-976/1344, ML-DSA-65/87). We also discuss the ECCG distinction between algorithms that require hybrid deployment (lattice-based mechanisms like ML-KEM and ML-DSA) and those where standalone use is acceptable (hash-based schemes like SLH-DSA, XMSS, LMS).

We use this to justify our choice of ML-KEM-768 in hybrid mode, which is consistent with the ECCG requirement that lattice-based KEMs should not be used standalone.

Task 3 – Implementation and Benchmarks

We implemented a TLS 1.3 client and server in C using OpenSSL 3.5.0 (built from source on Ubuntu 24.04 via Docker). We compare two configurations:

- **Classical:** X25519 key exchange, ECDSA P-256 authentication.
- **Hybrid:** X25519MLKEM768 key exchange, ECDSA P-256 authentication.

We measure three metrics per TLS session: handshake time (via `clock_gettime`), data transfer time, and handshake bytes on the wire (via a custom counting BIO filter). The client and server support multi-run mode (`-n N`) and CSV output for statistical analysis. A benchmark script computes mean and standard deviation across runs.

Preliminary results over 10 runs show the hybrid handshake is roughly 1.8× slower (∼2.5 ms vs. ∼1.4 ms) and exchanges 3.1× more bytes (∼3318 vs. ∼1054), while data transfer after the handshake is unaffected. A Dockerfile and benchmark scripts are provided for full reproducibility.

2 Questions for Feedback

1. **Hybrid combination choice:** We benchmark X25519MLKEM768 (classical X25519 vs. hybrid X25519+ML-KEM-768). Is one classical and one hybrid configuration sufficient, or should we also test a second hybrid (e.g., with ML-KEM-1024 or a different classical component)?

2. **Hybrid signatures in implementation:** Our TLS implementation uses hybrid KEM for key exchange but classical ECDSA for authentication, since OpenSSL does not support hybrid signature schemes in TLS 1.3. Is it sufficient to discuss hybrid signatures only in the ETSI theory section, or is there an expectation to also implement or demonstrate them outside of TLS?

3. **ETSI construction depth:** We present formulas and descriptions for each KEM and signature construction from ETSI TR 103 966. Should we also discuss the formal security models (e.g., IND-CCA2 for hybrid KEMs, EUF-CMA for hybrid signatures), or is the current level of detail appropriate?

4. **ECCG scope:** We cover the agreed algorithms and parameter levels from the ECCG document. Should we also discuss the ECCG migration timeline or transition recommendations in more depth, or is listing the algorithms and their hybrid/standalone status sufficient?

5. **Number of benchmark runs:** We currently report results over 10 runs. Would a larger number (e.g., 50 or 100 runs) be expected for the final report to reduce variance, or is 10 runs acceptable given the loopback setup?

6. **Network simulation (bonus):** We have implemented a `tc`-based network simulation script. If we include results under simulated conditions (e.g., added latency), how much analysis is expected – a brief comparison table, or a more detailed study?